HW3 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
1 using LinearAlgebra
```

1. Slacking off

Suppose A is invertible and $PA = LU$ is a computed LU factorization. We seek to solve the problem

$$(Ax - b)_i = 0 \text{ for all } i \neq k \text{ and } x_j = c$$

without refactoring A.

- Show that the desired problem can be rewritten as $\begin{bmatrix} A & e_k \\ e_j^T & 0 \end{bmatrix} \begin{bmatrix} x \\ r \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$.
- Assuming the problem in the previous part is well-posed and ignoring roundoff, write a short code `p1solve(A, F, k, j, b, c)` (where `F` is the LU factorization object) to compute x in $O(n^2)$ time using block GE. You may want to test your code with the following.

Part 1. The conditions mean $Ax = b + re_k$ for some scalar residual r (nonzero only at index k), and $x_j = c$. e_k is the k-th standard basis vector (1 at k, 0 elsewhere). This is equivalent to the augmented system: $\begin{bmatrix} A & e_k \\ e_j^T & 0 \end{bmatrix} \begin{bmatrix} x \\ r \end{bmatrix} = \begin{bmatrix} b \\ c \end{bmatrix}$. The first block row gives $Ax + re_k = b$, i.e., $Ax - b = -re_k$, so the residual is zero except at k. The second row gives $e_j^T x + 0 \cdot r = c$, i.e., $x_j = c$. Assuming the augmented matrix is invertible (which it is if the original problem is well-posed, as A is invertible and the constraint is consistent), this form allows us to apply block Gaussian elimination without full refactoring.

Part 2. see function p1solve(A, F, k, j, b, c) Let $u = A^{-1}e_k$ (solution to $Au = e_k$). Let $v = A^{-1}b$ (solution to $Av = b$). The Schur complement $0 - \frac{e_j^T e_k}{A} = -e_j^T A^{-1} e_k = -e_j^T u$. Plug into the constraint: $e_j^T x = e_j^T v - r e_j^T u = c$, so $r = \frac{c - e_j^T v}{S} = \frac{e_j^T v - c}{e_j^T u} = \frac{v_j - c}{u_j}$. Thus, $x = v - ru$.

```
p1solve (generic function with 1 method)
```

```
function p1solve(A, F, k, j, b, c)
    n = size(A, 1)
    ek = zeros(n)
    ek[k] = 1.0
    u = F \ ek
    v = F \ b
    if abs(u[j]) < 1e-8 # well-posed check
        error("Shur complement near to zero")
    end
    r = (v[j] - c) / u[j]
    x = v - r * u

    x
end
```

```
(5.69307e-15, 0.0)
```

```
let
    Atest = [1.0 2.0 3.0; 4.0 5.0 6.0; 7.0 8.0 10.0]
    b = [1.23; 0.5; 0.7]
    c = 3.14
    F = lu(Atest)
    x = p1solve(Atest, F, 1, 2, b, c)
    rr = Atest*x-b
    rr[1] = 0.0
    norm(rr)/norm(b), abs(x[2]-c)/c
end
```

```
let
    b = [1.23; 0.5; 0.7]
    c = 3.14
    F = lu(Atest)
    x = p1solve(Atest, F, 1, 2, b, c)
    rr = Atest*x-b
    rr[1] = 0.0
    norm(rr)/norm(b), abs(x[2]-c)/c
end
```

2. SDD redux

In lecture, we discussed *column* strictly diagonally dominant matrices (column SDD). There is an analogous property of *row* strict diagonal dominance (row SDD): A is strictly row diagonal dominant if $\sum_{j \neq i} |a_{ij}| < |a_{ii}|$ for each row i.

- Suppose A is strictly row diagonally dominant. Argue that if $(Ax)_i = 0$ for any nonzero x, then $|x_i| < \max_{j \neq i} |x_j|$ and therefore x_i cannot be the maximum magnitude element of x. *Hint*: Start by writing $x_i = -\sum_{j \neq i} a_{ij}/a_{ii} x_j$.
- Using the previous result, argue that if A is row SDD, then A^{-1} must be column SDD (and vice-versa).
- Also argue that if A is row (or column) SDD is written as a block 2-by-2 matrix, then A_{22} is also row (or column) SDD.
- Argue that therefore the Schur complements in a row SDD matrix are again row SDD.

1. Part 1. Suppose A is row SDD: $\sum_{j\neq i}|a_{ij}| < |a_{ii}|$ for each row i. Assume $(Ax)_i = 0$ for some nonzero x. From row i: $a_{ii}x_i + \sum_{j\neq i} a_{ij}x_j = 0 \implies x_i = -\frac{1}{} \sum_{j\neq i} a_{ij}x_j$. Taking absolute values: $|x_i| \leq \frac{\sum_{j\neq i}|a_{ij}|\cdot|x_j|}{a_{ii}} \leq \frac{\left(\sum_{j\neq i}|a_{ij}|\right)\max_{j\neq i}|x_j|}{|a_{ii}|} < \frac{|a_{ii}|\cdot\max_{j\neq i}|x_j|}{|a_{ii}|} = \max_{j\neq i}|x_j|$. Since the inequality is strict, $|x_i| < \max_{j\neq i}|x_j|$. Thus, x_i cannot be the maximum magnitude element of x.

2. Part 2. skip
3. Part 3. Let $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$. For rows in the lower block (indices for A_{22}), row SDD of A gives $|a_{ii}| > \sum_{j\neq i}|a_{ij}|$ over all off-diagonals in the row, including A_{21} entries. Thus, $\sum_{j\neq i, j\in A_{22}}|a_{ij}| = \sum_{\text{all off}}|a_{ij}| - \sum_{j\in A_{21}}|a_{ij}| < |a_{ii}| - 0 \leq |a_{ii}|$. Hence, A_{22} is row SDD.
4. Part 4. The Schur complement after eliminating the first block is $S = A_{22} - A_{21}A_{11}^{-1}A_{12}$. Since row SDD of A iff column SDD of A^T, and the Schur of A^T is S^T, $S^T = A_{22}^T - A_{12}^T(A_{11}^T)^{-1}A_{21}^T = (A_{22} - A_{21}A_{11}^{-1}A_{12})^T$. Thus, Schur complements preserve row SDD.

3. Complements to the KKT

Suppose

$$M = \begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix}$$

where $A \in \mathbb{R}^{n\times n}$ is positive definite and $B \in \mathbb{R}^{n\times k}, k < n$ has full column rank.

Show that we can factor M as $M = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}\begin{bmatrix} I & 0 \\ 0 & -I \end{bmatrix}\begin{bmatrix} L_{11}^T & L_{21}^T \\ 0 & L_{22}^T \end{bmatrix}$

and implement a code to complete this factorization given `F = cholesky(A)` (which gives $L11$). Your code should have the signature `L11, L21, L22 = kkt_ldlt(F, B)`; you may use the tester below to sanity check for correctness.

Hint: It helps to start with

$$M = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix}\begin{bmatrix} I & 0 \\ 0 & -S \end{bmatrix}\begin{bmatrix} L_{11}^T & L_{21}^T \\ 0 & I \end{bmatrix}$$

Given $M = \begin{bmatrix} A & B \\ B^T & 0 \end{bmatrix}$, with $A \succ 0$ (pos def), B full column rank. To show $M = LDL^T$, where $L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix}$, $D = \begin{bmatrix} I_n & 0 \\ 0 & -I_k \end{bmatrix}$, and the transpose is $\begin{bmatrix} L_{11}^T & L_{21}^T \\ 0 & L_{22}^T \end{bmatrix}$. Following the hint: $M = \begin{bmatrix} L_{11}L_{11}^T & L_{11}L_{21}^T \\ L_{21}L_{11}^T & L_{21}L_{21}^T - S \end{bmatrix}$

Check with M: $L_{11}L_{11}^T = A$. Since $A \succ 0$, L_{11} is the lower triangular Cholesky factor of A.
$L_{11}L_{21}^T = B$. $L_{21}L_{11}^T = B^T$ (consistent by transpose).
$L_{21}L_{21}^T - S = 0 \implies S = L_{21}L_{21}^T = B^T A^{-1} B$. Since $-S = -L_{21}L_{21}^T$ is negative definite, we can rewrite the lower-right as $L_{21}L_{21}^T + (-S) = L_{21}L_{21}^T - L_{22}L_{22}^T = 0$, where $L_{22}L_{22}^T = S$. Thus, L_{22} is the Cholesky factor of S.

Compute $M = LDL^T = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix} \begin{bmatrix} L_{11}^T & L_{21}^T \\ 0 & -L_{22}^T \end{bmatrix} = \begin{bmatrix} L_{11}L_{11}^T & L_{11}L_{21}^T \\ L_{21}L_{11}^T & L_{21}L_{21}^T - L_{22}L_{22}^T \end{bmatrix}$.

```
kkt_ldlt (generic function with 1 method)
```

```
function kkt_ldlt(F, B)
    L11 = F.L
    L21 = B' / L11'
    S = L21 * L21'
    L22 = cholesky(S).L
    return L11, L21, L22
end
```

```
6.679569284527639e-17
```

```
let
    A = [3.3 2.0 1.0; 2.0 3.1 1.0; 1.0 1.0 3.0]
    B = rand(3,2)
    M = [A B; B' 0*I]
    F = cholesky(A)
    L11, L21, L22 = kkt_ldlt(F, B)
    L = [L11 zeros(3,2); L21 L22]
    D = [I zeros(3,2); zeros(2,3) -I]
    norm(M - L*D*L')/norm(M)
end
```

```
let
    A = [3.3 2.0 1.0; 2.0 3.1 1.0; 1.0 1.0 3.0]
    B = rand(3,2)
    M = [A B; B' 0*I]
    F = cholesky(A)
    L11, L21, L22 = kkt_ldlt(F, B)
    L = [L11 zeros(3,2); L21 L22]
    D = [I zeros(3,2); zeros(2,3) -I]
    norm(M - L*D*L')/norm(M)
end
```